



Sanjivani Rural Education Society's
Sanjivani College of Engineering, Kopargaon-423 603
(An Autonomous Institute, Affiliated to Savitribai Phule Pune University, Pune)
NAAC 'A' Grade Accredited, ISO 9001:2015 Certified

Department of Computer Engineering

(NBA Accredited)

Subject- Object Oriented Programming (CO212)
Unit 1 – Fundamentals of OOP
Topic – 1.11 Constructors and Destructors

Prof. V.N.Nirgude
Assistant Professor

E-mail :

nirgudevikascomp@sanjivani.org.in

contact No: 9975240215



CONSTRUCTOR

A member function with the same name as its class name is called Constructor. It is used to initialize the object of that class with a legal initial value.

Example :

```
class student
{
private:
    int rollno;
    float marks;
public:
    student( )           //Constructor
    {
        rollno=0; marks=0.0;
    }
};
```



CONSTRUCTOR CALLING

A constructor can be called in two ways: implicitly and explicitly.

```
class student
{
    private:
        int rollno;
        float marks;
    public:
        student( )           //Constructor
        { rollno=0; marks=0.0; }
};
void main()
{
    student s1; //implicit call
}
```

IMPLICIT CALL OF CONSTRUCTOR

```
class student
{
    private:
        int rollno;
        float marks;
    public:
        student( )           //Constructor
        { rollno=0; marks=0.0; }
};
void main()
{
    student s1 = student(); //explicit call
}
```

EXPLICIT CALL OF CONSTRUCTOR



FEATURES OF CONSTRUCTOR

- ☐ Constructor has the same name as its class.
- ☐ Constructor does not have any return type not even void.
- ☐ Constructor have to be defined only in the public section of the class.
- ☐ Constructor is called automatically when the object of the class is created.



TYPES OF CONSTRUCTOR

1. **Default Constructor:** A constructor that accepts no parameter is called Default Constructor.
2. **Parameterized Constructor:** A constructor that accepts parameters for its invocation is known as parameterized Constructor.
3. **Copy Constructor:** A copy constructor is a special constructor used to create a new object as a copy of an existing object.



DEFAULT CONSTRUCTOR

- ❑ Default constructor takes no argument.
- ❑ It is used to initialize an object of a class with legal initial values.

Example :

```
class student
{
    private:
        int rollno;
        float marks;
    public:
        student( )           // Default Constructor
        { rollno=0; marks=0.0; }
        void display();
        void enter();
};

void main()
{
    student s1; // default constructor called
}
```



PARAMETERISED CONSTRUCTOR

- ❑ Parametrized constructor accepts arguments.
- ❑ It initializes the data members of the object with the arguments passed to it.
- ❑ When the object s1 is created, default constructor is called and when the objects s2 is created, parametrized constructor is called.
When the object s3 is created, again the parametrized constructor is called but this is explicit call of parametrized constructor.

```
class stud
{
    private:
        int rollno;
        char name[20];
        float marks;
    public:
        stud()
        { rollno=0; marks=0.0; }

        stud(int roll, char nam[20],float
mar)
        { rollno=roll;
          strcpy(name, nam);
          marks =mar;
        }
}
```

```
void display();
void enter();
};

void main()
{
    stud s1;
    stud s2(1, "RAM", 350);
    stud s3=stud(2,"NEHA", 400);
}
```



COPY CONSTRUCTOR

- ❑ Copy constructor accepts object as an argument.
- ❑ It creates a new object and initializes its data members with data members of the object passed to it as an argument.

Syntax : `class_name( class_name & object_name)`
{

}

COPY CONSTRUCTOR EXAMPLE

```
class student
{
    private:
        int rollno;
        char name[20];
        float marks;
    public:
        student()
        {
            rollno=0; marks=0.0;
        }
        student(int roll, char nam[20], float mar)
        {
            rollno=roll;
            strcpy(name, nam);
            marks =mar;
        }
}
```

```
    student( student &s)
    {
        rollno= s.rollno;
        strcpy(name, s.name);
        marks =s.marks;
    }
};

int main()
{
    student s1;
    student s2( 1, "RAJESH", 450);
    student s3(s2);
    student s4(2, "MOHIT",320);
    student s5=s4;
}
```



CONSTRUCTOR OVERLOADING

- ❑ Defining more than one constructors in a single class provided they all differ in either type of arguments or number of argument is called constructor overloading.
- ❑ In previous example, in class student, there are three constructors:
1 default constructor, 1 parametrized constructors and 1 copy constructor.
- ❑ We can define as many parametrized constructors as we want in a single class provided they all differ in either number or type of arguments.



DESTRUCTOR

- ❑ A destructor is a member function whose name is same as the class name and is preceded by tilde symbol("~").
- ❑ It is automatically called by the compiler when an object goes out of scope.
- ❑ Destructors are usually used to deallocate memory which was allocated to object when the object is destroyed.
- ❑ **Note:** In this program first the object s1 is created. At that time constructor is called. When the program ends, s1 will also go out of scope. At that time destructor is called.

Example :

```
class student
{ private:
    int rollno;
    float marks;
public:
    student( )
    { rollno=0; marks=0.0; }
    ~student()
    { }
};

void main()
{
    student s1;
}
```



FEATURES OF DESTRUCTOR

- ☐ It has the same name as that of class name and preceded by tilde sign(~).
- ☐ It is automatically called by the compiler when an object goes out of scope.
- ☐ Destructors are usually used to deallocate memory when the object goes out of scope.
- ☐ Destructor is always declared in the public section of the class.
- ☐ We can have only one destructor in one class.

Thank You..